Module Code	IY4113
Group	B
Module Title	Practical assignment part 1: Java Fundamentals
Assessment Type	Report
Module Tutor Name	Jonathan Shore
Student ID Number	P510008
Assessment Window	Final submission: 16/02/2026- 22/02/2026

☒ *I confirm that this assignment is my own work. Where I have referred to academic sources, I have provided in-text citations and included the sources in the final reference list.*

☒ *Where I have used AI, I have cited and referenced appropriately.*

KAPLAN INTERNATIONAL PATHWAYS

TABLE OF CONTENTS

(Page 2)

Introduction 3

Analysis 4

2.1 Functional Requirements 4

2.2 Non - Functional Requirements 5

2.3 Input – Process - Output Table 6

Algorithm Design 8

3.1 Main Menu Flowchart 8

3.2 Add Journey Flowchart 9

3.3 Calculate Fare Flowchart 10

3.4 Filter Journeys Flowchart 11

3.5 Remove Journey Flowchart 12

3.6 Class Diagram 13

Testing 14

4.1 Black Box Testing 14

4.2 Scanner Input Testing 16

Evaluation 18

Code Listing 20

References 25

Appendices 27

Appendix A - Milestone 1

Appendix B - Milestone 2

Appendix C - Milestone 3

Appendix D - Milestone 4

Introduction

This project involves the design and development of a console - based Java application called CityRide Lite, which is used to record and manage public transport journeys for a single day. The aim of the system is to allow users to input journey details such as travel zones, passenger type, and time band, and then calculate fares based on a fixed dataset that includes base fares, discounts, and daily caps. The system provides a range of features including adding and storing journeys, filtering and removing journeys, and generating daily summary reports. All interactions with the system are performed through a text - based menu and stores all journey data in memory for the duration of the session. This makes the program simple to use while still demonstrating important programming concepts such as input validation, control and class structures, collections, and the OOP design. The development of this system required the application of key software engineering principles, including requirements analysis, algorithm design using flowcharts, object - oriented programming (OOP), and testing of the program. Throughout the development process, the system was built progressively across multiple milestones, allowing for reflection, refinement, and improvement. This results in a final completion that includes program code, design documentation, testing proof, and an evaluation of how well the system meets the original requirements.

This report will firstly present a detailed analysis of the program's functional and non - functional requirements. This section includes an Input – Process - Output (IPO) table used to define the boundaries of the system and the logic required for fare calculations. Secondly, it will outline the structural design of the program through the use of flowcharts to visualize logic flow and class diagrams to illustrate the Object - Oriented Programming Structure. This is followed by a detailed section on system testing, specifically focusing on black - box testing of the Scanner class - based input methods to ensure data integrity, program stability and system functionality. Finally, the report will provide a critical evaluation of the project, assessing how effectively the final application meets the specified brief, before concluding with the full source code listing, references, and relevant appendices.

Analysis

The CityRide Lite system was developed based on a detailed specification according to the functional and non - functional requirements outlined in the assignment brief. The program is required to operate as a console - based, in memory application that records public transport journeys for a single day. The main objective is to simulate a simplified public transport fare system that calculates journey costs based on predefined rules. No file storage is used, and all data is reset when the program terminates.

Functional Requirements

Functional requirements include;

- Displaying a looping main menu which allow user selection.
- Adding journeys with validated input (date, zones, passenger type, time band).
- Calculate fare based on zones crossed, discounts, and daily caps.
- Determining base fares using the provided dataset.
- Applying passenger discounts.
- Enforcing daily caps per passenger type.
- Storing journeys in memory with unique IDs
- Listing journeys recorded for the day.

- Filter journeys based on passenger type.
- Remove a journey using its ID as validation.
- Reset all stored data for a new day.
- Generating daily summaries
- Exit the program safely.

Non-Functional Requirements

Non - Functional requirements include;

- The system must use object-oriented programming principles.
- Input validation and error handling to prevent invalid or incorrect data.
- The program must run efficiently in a console - based interface.
- Clear program structure and readability.
- The system must be easy to use through a structured menu interface.

Input / Process / Output Design

The Input – Process - Output (IPO) table was developed to clearly identify:

- The data required from the user
- The internal processing performed by the system
- The output generated for each feature

Each feature is initiated by user input (e.g., menu choice, journey details, journey ID), ensuring that the system remains user driven as required by the assignment brief.

Feature / Task	Inputs	Processing (what the system does)	Outputs
Start program	None (program launch)	Initialise data structures and running totals	Main menu shown
Display menu (FR1)	User menu choice (1 - 5) (int)	Validate choice and route to correct function	Selected screen displayed
Add journey (FR2)	Date(String), from zone(int), to zone(int), passenger type(String), time band(String) - peak/off-peak	Validate inputs, calculate zones crossed, lookup fare, apply discount and daily cap	Journey added confirmation
Calculate fare (FR3)	From zone (int), to zone(int), time band (String), passenger type (String)	Calculates zones crossed, get base fare, apply discount and daily cap	Final charged fare
Store journey (FR4)	Journey details (Data Structure)	Create Journey object and assign ID	Journey stored in memory
List journeys	View Request (Boolean: Y/N)	Loop through journey list	Display all journeys
Filter journeys (FR5)	Filter Criteria (Type: String, Value: String/Integer)	Search journey list	Filtered journey list
Remove journey (FR6)	Journey ID (Integer)	Validate ID, remove journey, recalculate totals	Confirmation or error
Daily summary (FR7)	Summary Request (Boolean: Y/N)	Calculate totals, averages, max fare	Summary report
Reset day	Confirmation (Boolean: Y/N)	Clear all lists and totals	System reset message
Exit	Menu choice – 5 : Exit Command (int)	Terminate program	Goodbye message

Algorithm Design

The algorithm design for CityRide Lite was developed using structured flowcharts and a class diagram to visually demonstrate the program logic and object - oriented structure. The

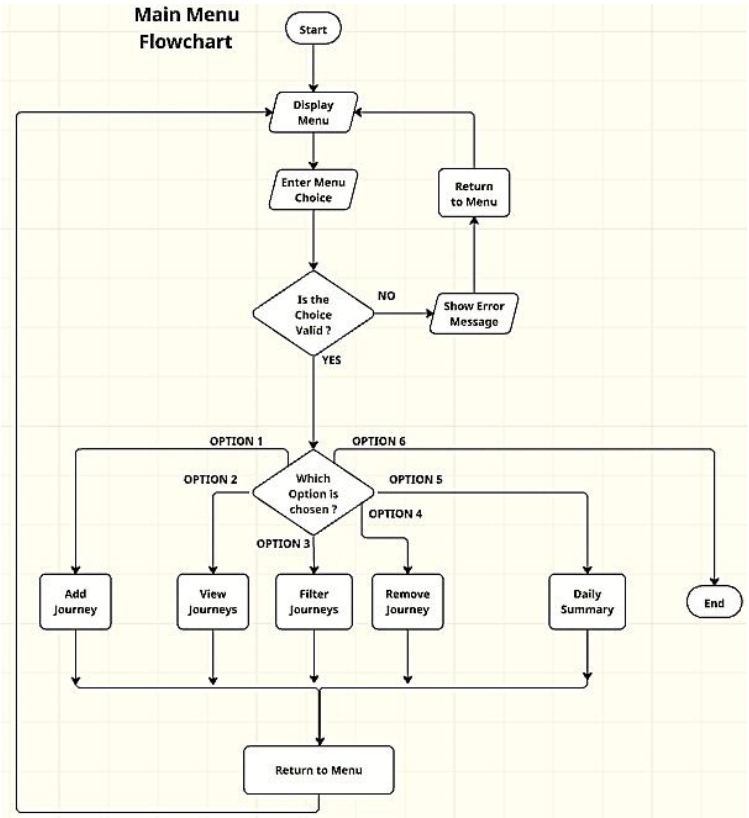
system was decomposed into smaller functional components to improve clarity and maintainability.

Main Menu Flowchart

The main menu flowchart represents the overall control structure of the program. The system begins execution in the main method and enters a loop that continues until the user selects the Exit option. When the program starts, the menu options are displayed and the user is prompted to enter a choice. The input is validated to ensure it is within the permitted range.

If the input is invalid, an error message is displayed and the system returns to the menu. If valid, the corresponding method is executed (Add Journey, Filter Journeys, Remove Journey, etc.). After completing the selected function, the program returns to the main menu unless the user selects the Exit option, which terminates the program.

This flowchart ensures structured program control and continuous user interaction.



Add Journey Flowchart

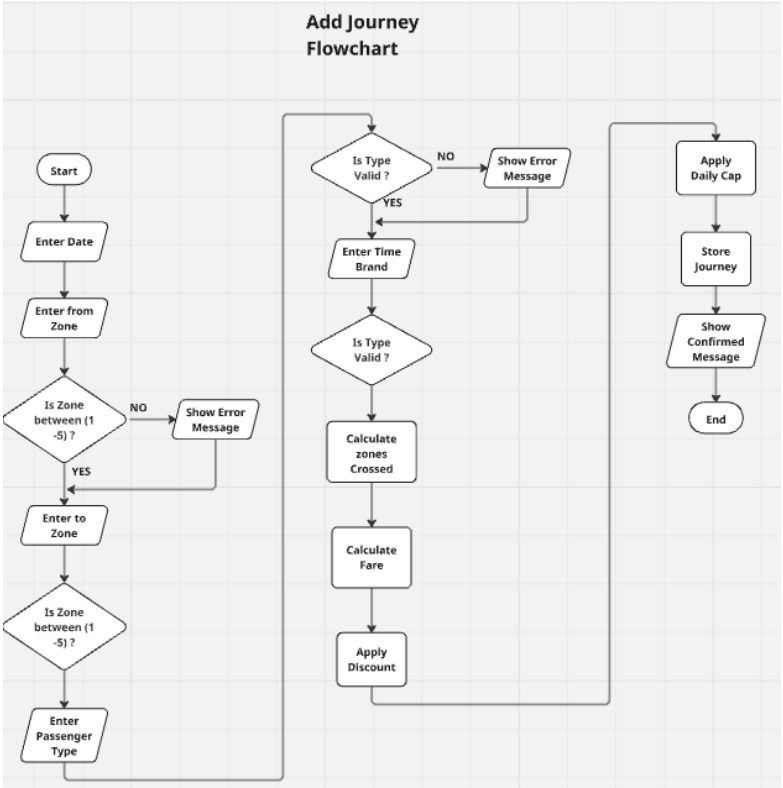
The Add Journey process is the most detailed functional component of the system. The algorithm follows these steps:

- Prompt user for from zone
- Validate zone range (1–5)
- Prompt user for to zone
- Validate zone range
- Prompt for passenger type
- Validate passenger type

- Prompt for time band
- Validate time band
- Calculate zones crossed
- Retrieve base fare from dataset
- Apply discount rate
- Check daily cap
- Adjust fare if necessary
- Create Journey object
- Store journey in memory
- Display confirmation

A new Journey object is created and stored in memory through the JourneyManager class. A confirmation message is displayed before returning to the main menu.

This Add Journey flowchart represents the process of recording a new journey.



Calculate Fare Flowchart

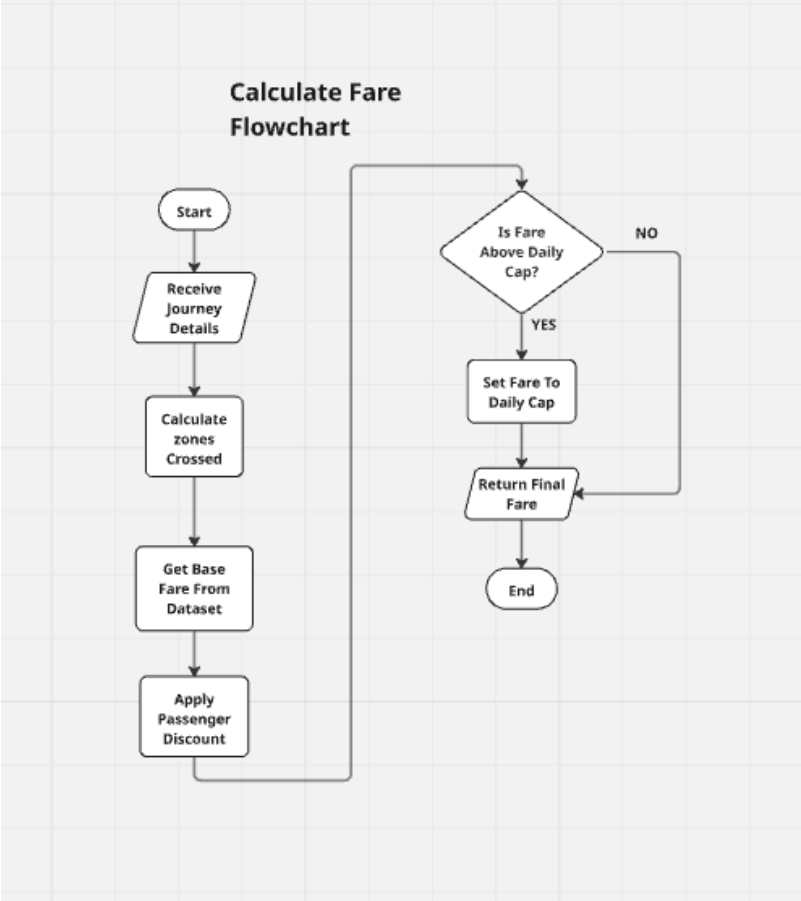
The Calculate Fare flowchart focuses specifically on fare computation logic. It separates business logic from user interaction.

The process includes:

- Receiving validated zone and passenger data
- Calculating zones crossed
- Looking up base fare from CityRideDataset
- Applying discount percentage
- Checking whether the fare exceeds the daily cap

- Adjusting the fare if necessary
- Returning the final charged fare

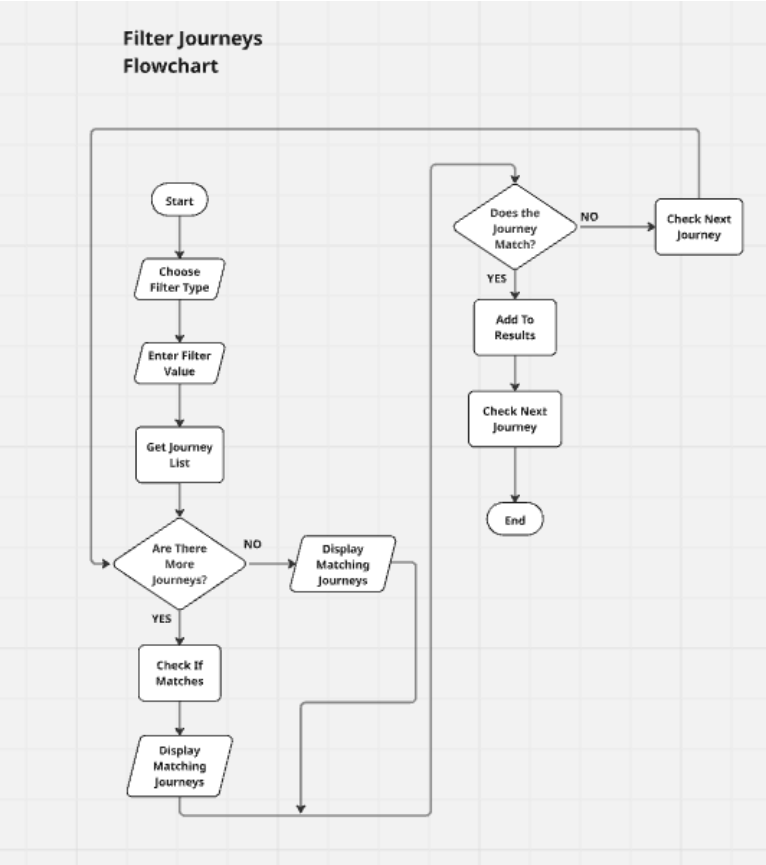
Separating this logic into its own flowchart improves modularity and supports reusable fare calculations within the program.



Filter Journeys Flowchart

The Filter Journeys flowchart demonstrates how the system searches stored journeys based on specific criteria. The journey list is then searched using a loop. Matching journeys are displayed. If no matches are found, an appropriate message is shown. Control then returns to the main menu.

The Filter Journeys flowchart allows users to search for journeys based on passenger type. The system prompts the user for the filter value and validates the input.



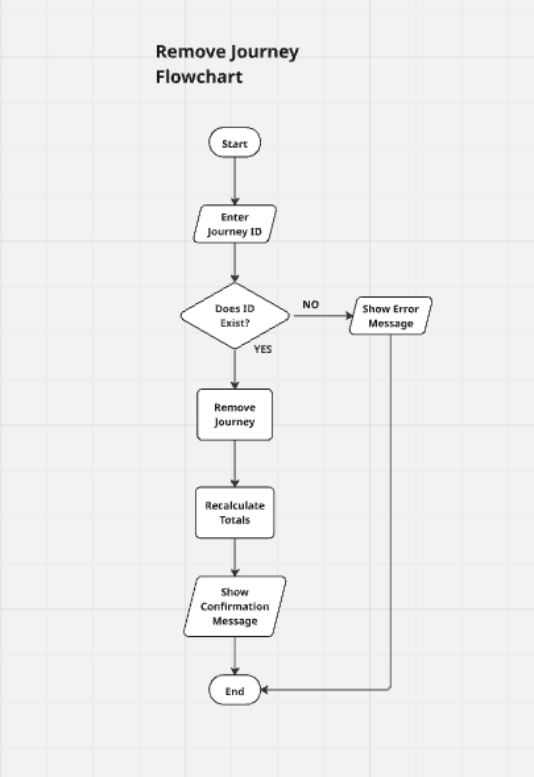
Remove Journey Flowchart

The Remove Journey flowchart allows a user to delete a stored journey. The system prompts the user to enter a Journey ID. The ID is validated and the journey list is searched. If the journey is found, it is removed from memory and totals are recalculated where necessary. A confirmation message is displayed. If the ID does not exist, an error message is shown.

This ensures controlled data management within the system.

The process includes:

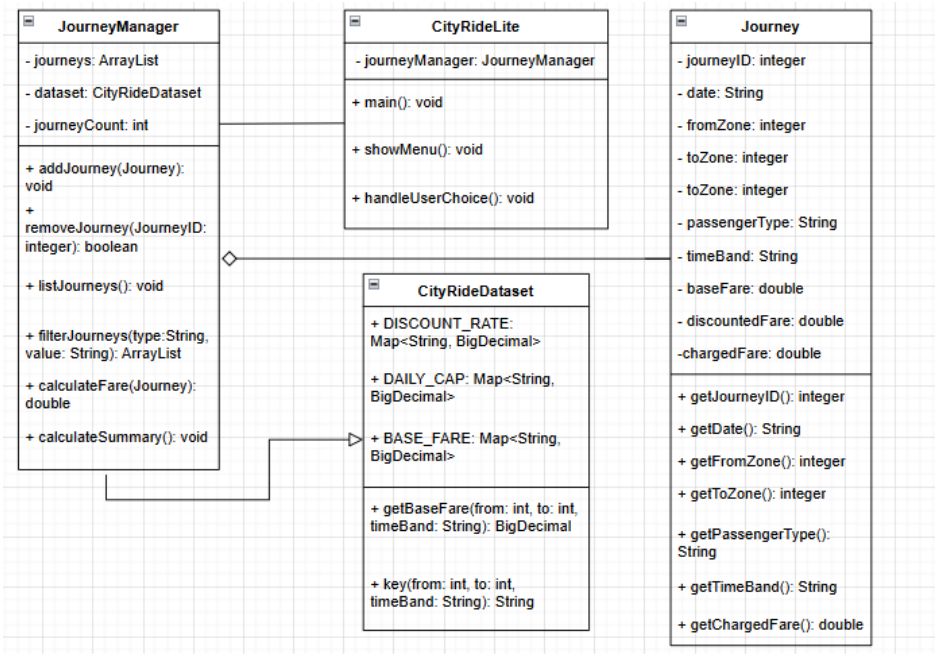
- Entering Journey ID
- Validating that the ID exists
- Removing the journey from storage
- Recalculating totals
- Displaying confirmation or error message



Class Diagram Design Overview

The class diagram demonstrates the object - oriented structure of the system. The application consists of four primary classes:

- CityRideLite (Controls program execution and user interaction)
- JourneyManager (manages journey collection and fare logic)
- Journey (represents individual journey data)
- CityRideDataset (provides fixed fare and discount data)



The relationships are as follows:

The CityRideLite class has an association relationship with JourneyManager, as it calls its methods. The JourneyManager class has an aggregation relationship with Journey, as it stores multiple Journey objects. The JourneyManager also depends on the CityRideDataset class for fare calculation data.

- CityRideLite → uses → JourneyManager
- JourneyManager → aggregates → Journey (1 to many)
- JourneyManager → depends on → CityRideDataset

Encapsulation is applied by keeping attributes private and exposing only necessary methods.

Testing

Black box testing was conducted on all user input methods, particularly only methods for the Scanner class. Black Box Testing will be based on the Functional Requirements.

So we test:

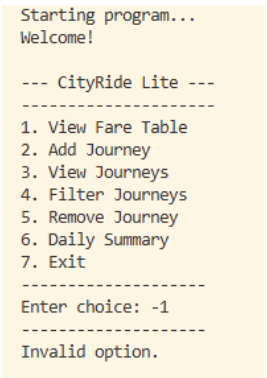
- Menu input
- Zone input
- Passenger type input
- Time band input
- Remove journey input
- Filter input

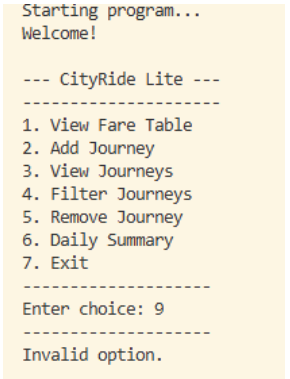
When we do black box testing we test inputs and outputs and not the internal code logic.

Black Box Testing Table

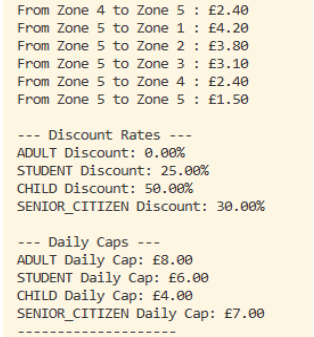
Functional Requirement: Display Menu (FR1)

Test No.	Input	Expected Output	Actual Output	Result
1	2	Add Journey screen shown	Add Journey prompts displayed	Pass
2	1	Fare table displayed	Fare table displayed	Pass
3	9	Invalid option message	"Invalid option" shown	Pass
4	-1	Invalid option message	"Invalid option" shown	Pass









Functional Requirement: Display Menu (FR1)

Test No.	Input	Expected Output	Actual Output	Result
5	Valid zones (1-3), ADULT, PEAK	Journey stored	Journey stored successfully	Pass
6	Zone 6	Error message	Validation message shown	Pass
7	Invalid passenger type	Error message	Input rejected	Pass

Functional Requirement: Calculate Fare (FR3)

Test No.	Input	Expected Output	Actual Output	Result
8	Zone 1-3 PEAK ADULT	Correct base fare	Correct fare displayed	Pass
9	STUDENT discount	Fare reduced 25%	Correct reduced fare	Pass
10	Daily cap exceeded	Fare capped	Cap applied correctly	Pass

Functional Requirement: Remove Journey (FR6)

Test No.	Input	Expected Output	Actual Output	Result
11	Valid ID	Journey removed	Removal confirmation	Pass
12	Invalid ID	Error message	ID not found message	Pass

Functional Requirement: Filter Journey (FR5)

Test No.	Input	Expected Output	Actual Output	Result
13	Filter ADULT	Only adult journeys shown	Correct filtering	Pass
14	Filter with no matches	No journeys found	Correct message shown	Pass

```
--------------------
Enter choice: 5

--------------------
Passenger Name: Deborah
Journey ID: 1
From Zone: 3
To Zone: 2
Passenger Type: STUDENT
Zones Crossed: 1
Time Band: PEAK
Fare: £2.3250
--------------------
Enter Journey ID to remove: 1
Journey removed.

--- CityRide Lite ---
--------------------
1. View Fare Table
2. Add Journey
3. View Journeys
4. Filter Journeys
5. Remove Journey
6. Daily Summary
7. Exit
--------------------
Enter choice: 5
No journeys available to remove.
```

```
--- CityRide Lite ---
--------------------
1. View Fare Table
2. Add Journey
3. View Journeys
4. Filter Journeys
5. Remove Journey
6. Daily Summary
7. Exit
--------------------
Enter choice: 2
Enter passenger name: Deborah
--------------------
Enter from zone (1-5): 3
--------------------
Enter to zone (1-5): 2
--------------------
ADULT
STUDENT
CHILD
SENIOR_CITIZEN
Passenger type: student
--------------------
PEAK
OFF_PEAK
Time band: peak

--------------------
Journey added successfully.
Fare charged: £2.3250
--------------------
```

```
--------------------
Enter Journey ID to remove: 1
Journey removed.

--- CityRide Lite ---
--------------------
1. View Fare Table
2. Add Journey
3. View Journeys
4. Filter Journeys
5. Remove Journey
6. Daily Summary
7. Exit
--------------------
Enter choice: 5
No journeys available to remove.
```

Evaluation

The CityRide Lite system successfully meets the majority of the functional requirements outlined in the assignment brief. The program allows users to add, store, filter, remove, and summarise journeys through a structured console interface. Fare calculations are correctly performed using the provided dataset, including application of base fares, passenger discounts, and daily caps. The system also demonstrates effective use of object-oriented principles such as encapsulation, separation of concerns, and class interaction.

It successfully:

- Implements full journey storage
- Applies discounts correctly
- Enforces daily caps accurately
- Validates inputs robustly
- Provides detailed summaries
- Prevents invalid removals

One of the strengths of the program is its modular design. The use of separate classes (**CityRideLite**, **JourneyManager**, **Journey**, and **CityRideDataset**) improves maintainability and readability. The inclusion of enums and BigDecimal ensures accurate financial calculations and prevents floating - point rounding errors. The addition of the fare table viewing feature also improves transparency and usability.

However, there are some limitations. The system stores journeys in memory only, meaning data is lost when the program exits. The Input validation could be further strengthened to prevent unexpected crashes from invalid data types. Additionally, the console - based interface limits user experience compared to a graphical interface (GUI).

In future improvements, the system could be extended by:

- Saving journeys to a file or database
- Implementing a graphical user interface (GUI)
- Improving input validation with exception handling
- Improving reporting with formatted summary tables

Overall, the program effectively satisfies the core requirements and demonstrates strong understanding of Java programming concepts, algorithm design, and structured software development.

Code Listing

CityRideLite.java

```
import java.util.Scanner;
import java.math.BigDecimal;

public class CityRideLite {

    //oracle
    private static void viewFareTable() {
```

```
        System.out.println("\n--- Fare Table ---");

        for (CityRideDataset.TimeBand band :
CityRideDataset.TimeBand.values()) {

            System.out.println("\nTime Band: " + band);
            System.out.println("----------------------------------");

            for (int from = CityRideDataset.MIN_ZONE; from <=
CityRideDataset.MAX_ZONE; from++) {
                for (int to = CityRideDataset.MIN_ZONE; to <=
CityRideDataset.MAX_ZONE; to++) {

                    BigDecimal fare = CityRideDataset.getBaseFare(from, to,
band);

                    System.out.println("From Zone " + from +
                            " to Zone " + to +
                            " : £" + fare);
                }
            }
        }

        System.out.println("\n--- Discount Rates ---");
        for (CityRideDataset.PassengerType type :
CityRideDataset.PassengerType.values()) {
            System.out.println(type + " Discount: "
                    + CityRideDataset.DISCOUNT_RATE.get(type).multiply(new
BigDecimal("100"))
                    + "%");
        }

        System.out.println("\n--- Daily Caps ---");
        for (CityRideDataset.PassengerType type :
CityRideDataset.PassengerType.values()) {
            System.out.println(type + " Daily Cap: £"
                    + CityRideDataset.DAILY_CAP.get(type));
        }
    }

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);
        JourneyManager journeyManager = new JourneyManager();
        int choice;

            System.out.println("\nStarting program...\n");
            System.out.print("====================\n");
```

```
            System.out.println("     WELCOME");
            System.out.println("       TO");
        do {
            System.out.println("  CITYRIDE LITE!");
            System.out.print("====================\n");
            System.out.println("1. View Fare Table");
            System.out.println("2. Add Journey");
            System.out.println("3. View Journeys");
            System.out.println("4. Filter Journeys");
            System.out.println("5. Remove Journey");
            System.out.println("6. Daily Summary");
            System.out.println("7. Exit");
            System.out.print("--------------------\n");
            System.out.print("Enter choice: ");
            choice = scanner.nextInt();

            if (choice == 1) {
                viewFareTable();
            }

            else if (choice == 2) {

                int fromZone;
                int toZone;
                System.out.print("--------------------\n");
                System.out.print("Enter passenger name: ");
                scanner.nextLine();
                String passengerName = scanner.nextLine();

                do {
                    System.out.print("--------------------\n");
                    System.out.print("Enter from zone (1-5): ");
                    fromZone = scanner.nextInt();
                } while (fromZone < 1 || fromZone > 5);

                do {
                    System.out.print("--------------------\n");
                    System.out.print("Enter to zone (1-5): ");
                    toZone = scanner.nextInt();
                } while (toZone < 1 || toZone > 5);

                System.out.print("--------------------\n");
                System.out.print("ADULT\nSTUDENT\nCHILD\nSENIOR_CITIZEN\n");
                System.out.print("--------------------\n");
                System.out.print("Passenger type: ");
                CityRideDataset.PassengerType passengerType =
                        CityRideDataset.PassengerType.valueOf(
                                scanner.next().toUpperCase()
```

```
                        );

                System.out.print("--------------------\n");
                System.out.print("PEAK\nOFF_PEAK\n");
                System.out.print("--------------------\n");
                System.out.print("Time band: ");
                CityRideDataset.TimeBand timeBand =
                        CityRideDataset.TimeBand.valueOf(
                                scanner.next().toUpperCase()
                        );

                journeyManager.addJourney(passengerName, fromZone, toZone,
passengerType, timeBand);

            } else if (choice == 3) {
                journeyManager.listJourneys();

            } else if (choice == 4) {
                System.out.print("--------------------\n");
                System.out.print("Filter by passenger type: ");
                CityRideDataset.PassengerType type =
                        CityRideDataset.PassengerType.valueOf(
                                scanner.next().toUpperCase()
                        );
                journeyManager.filterByPassengerType(type);

            } else if (choice == 5) {
                if (journeyManager.hasJourneys()) {
                    journeyManager.listJourneys();
                    System.out.print("Enter Journey ID to remove: ");
                    int id = scanner.nextInt();
                    journeyManager.removeJourney(id);
                } else {
                    System.out.print("--------------------\n");
                    System.out.println("No journeys available to remove.");
                    System.out.print("--------------------\n");
            }
            }

            else if (choice == 6) {
                journeyManager.showSummary();
            }

            else if (choice == 7) {
                System.out.print("--------------------\n");
                System.out.println("Goodbye!\n");
                System.out.println("Exiting program...");
```

```
            } else {
                System.out.print("--------------------\n");
                System.out.println("Invalid option.\n");
                System.out.print("--------------------\n");

            }

        } while (choice != 7);

        scanner.close();
    }
}
```

Journey.java

```
import java.math.BigDecimal;

public class Journey {

    private String passengerName;
    private int id;
    private int fromZone;
    private int toZone;
    private CityRideDataset.PassengerType passengerType;
    private CityRideDataset.TimeBand timeBand;
    private BigDecimal fare;

    public Journey(String passengerName, int id, int fromZone, int toZone,
                   CityRideDataset.PassengerType passengerType,
                   CityRideDataset.TimeBand timeBand,
                   BigDecimal fare) {

        this.passengerName = passengerName;
        this.id = id;
        this.fromZone = fromZone;
        this.toZone = toZone;
        this.passengerType = passengerType;
        this.timeBand = timeBand;
        this.fare = fare;
    }

    //geeksforgeeks
    public int getId() {
        return id;
    }

    public int getFromZone() {
        return fromZone;
```

```
    }

    public int getToZone() {
        return toZone;
    }

    public CityRideDataset.PassengerType getPassengerType() {
        return passengerType;
    }

    public int getZonesCrossed() {return Math.abs(toZone - fromZone); }

    public CityRideDataset.TimeBand getTimeBand() {
        return timeBand;
    }

    public BigDecimal getFare() {
        return fare;
    }

    public void setFare(BigDecimal fare) {
        this.fare = fare;
    }

    public void displayJourney() {
        System.out.print("--------------------\n");
        System.out.println("Passenger Name: " + passengerName);
        System.out.println("Journey ID: " + id);
        System.out.println("From Zone: " + fromZone);
        System.out.println("To Zone: " + toZone);
        System.out.println("Passenger Type: " + passengerType);
        System.out.println("Zones Crossed: " + getZonesCrossed());
        System.out.println("Time Band: " + timeBand);
        System.out.println("Fare: £" + fare);
        System.out.print("--------------------\n");
    }
}
```

JourneyManager.java

```
import java.math.BigDecimal;
import java.math.RoundingMode;

public class JourneyManager {

    private Journey[] journeys;
    private int journeyCount;
    private int nextId;
```

```
//geeksforgeeks
    public JourneyManager() {
        journeys = new Journey[5];
        journeyCount = 0;
        nextId = 1;
    }
    public boolean hasJourneys() {
        return journeyCount > 0;
    }
    //oracle
    public void addJourney(String passengerName, int fromZone, int toZone,
                           CityRideDataset.PassengerType passengerType,
                           CityRideDataset.TimeBand timeBand) {

        BigDecimal baseFare =
                CityRideDataset.getBaseFare(fromZone, toZone, timeBand);

        BigDecimal discountRate =
                CityRideDataset.DISCOUNT_RATE.get(passengerType);

        BigDecimal discount =
                baseFare.multiply(discountRate);

        BigDecimal finalFare =
                baseFare.subtract(discount);

        BigDecimal totalSoFar = getTotalForPassenger(passengerType);
        BigDecimal cap = CityRideDataset.DAILY_CAP.get(passengerType);

        if (totalSoFar.add(finalFare).compareTo(cap) > 0) {
            finalFare = cap.subtract(totalSoFar);
            if (finalFare.compareTo(BigDecimal.ZERO) < 0) {
                finalFare = BigDecimal.ZERO;
            }
        }

        Journey journey = new Journey(
                passengerName,
                nextId,
                fromZone,
                toZone,
                passengerType,
                timeBand,
                finalFare
        );

        journeys[journeyCount] = journey;
```

```
        journeyCount++;
        nextId++;

        System.out.print("--------------------\n");
        System.out.println("Journey added successfully.");
        System.out.println("Fare charged: £" + finalFare);
        System.out.print("--------------------\n");
    }

    private BigDecimal getTotalForPassenger(CityRideDataset.PassengerType
type) {
        BigDecimal total = BigDecimal.ZERO;
        for (int i = 0; i < journeyCount; i++) {
            if (journeys[i].getPassengerType() == type) {
                total = total.add(journeys[i].getFare());
            }
        }
        return total;
    }

    public void listJourneys() {
        if (journeyCount == 0) {
            System.out.print("--------------------\n");
            System.out.println("No journeys recorded.");
            System.out.print("--------------------\n");
        } else {
            for (int i = 0; i < journeyCount; i++) {
                journeys[i].displayJourney();
            }
        }
    }

    public void removeJourney(int id) {
        boolean found = false;

        for (int i = 0; i < journeyCount; i++) {
            if (journeys[i].getId() == id) {
                found = true;

                for (int j = i; j < journeyCount - 1; j++) {
                    journeys[j] = journeys[j + 1];
                }
                journeyCount--;
                System.out.print("--------------------\n");
                System.out.println("Journey removed.");
                System.out.print("--------------------\n");
                break;
            }
```

```
        }

        if (!found) {
            System.out.print("--------------------\n");
            System.out.println("Journey ID not found.");
            System.out.print("--------------------\n");
        }
    }

    public void filterByPassengerType(CityRideDataset.PassengerType type) {
        boolean found = false;

        for (int i = 0; i < journeyCount; i++) {
            if (journeys[i].getPassengerType() == type) {
                journeys[i].displayJourney();
                found = true;
                System.out.println();
            }
        }

        if (!found) {
            System.out.print("--------------------\n");
            System.out.println("No journeys found for that passenger type.");
            System.out.print("--------------------\n");
        }
    }

    public void showSummary() {
        if (journeyCount == 0) {
            System.out.print("--------------------\n");
            System.out.println("No journeys to summarise.");
            System.out.print("--------------------\n");
            return;
        }

        //geeksforgeeks
        BigDecimal total = BigDecimal.ZERO;
        BigDecimal max = BigDecimal.ZERO;
        int peakCount = 0;
        int offPeakCount = 0;

        for (int i = 0; i < journeyCount; i++) {
            total = total.add(journeys[i].getFare());

            if (journeys[i].getFare().compareTo(max) > 0) {
                max = journeys[i].getFare();
            }
```

```
            if (journeys[i].getTimeBand() == CityRideDataset.TimeBand.PEAK) {
                peakCount++;
            } else {
                offPeakCount++;
            }
        }

        BigDecimal average =
                total.divide(BigDecimal.valueOf(journeyCount), 2,
RoundingMode.HALF_UP);

        System.out.println("-- Daily Summary --");
        System.out.print("--------------------\n");
        System.out.println("Total journeys: " + journeyCount);
        System.out.println("Total cost: £" + total);
        System.out.println("Average cost: £" + average);
        System.out.println("Most expensive journey: £" + max);
        System.out.println("Peak journeys: " + peakCount);
        System.out.println("Off-peak journeys: " + offPeakCount);
        System.out.print("--------------------\n");
    }
}
```

CityRideDataset.java

```
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.HashMap;
import java.util.Map;

final class CityRideDataset {

    private CityRideDataset() {}

    public static final int MIN_ZONE = 1;
    public static final int MAX_ZONE = 5;

    public enum TimeBand {
        PEAK,
        OFF_PEAK
    }

    public enum PassengerType {
        ADULT,
        STUDENT,
        CHILD,
        SENIOR_CITIZEN
    }
```

```
    public static final Map<PassengerType, BigDecimal> DISCOUNT_RATE = Map.of(
            PassengerType.ADULT, new BigDecimal("0.00"),
            PassengerType.STUDENT, new BigDecimal("0.25"),
            PassengerType.CHILD, new BigDecimal("0.50"),
            PassengerType.SENIOR_CITIZEN, new BigDecimal("0.30")
    );

    public static final Map<PassengerType, BigDecimal> DAILY_CAP = Map.of(
            PassengerType.ADULT, new BigDecimal("8.00"),
            PassengerType.STUDENT, new BigDecimal("6.00"),
            PassengerType.CHILD, new BigDecimal("4.00"),
            PassengerType.SENIOR_CITIZEN, new BigDecimal("7.00")
    );

    public static final Map<String, BigDecimal> BASE_FARE = buildBaseFare();

    public static BigDecimal getBaseFare(int fromZone, int toZone, TimeBand
timeBand) {
        return BASE_FARE.get(key(fromZone, toZone, timeBand));
    }

    public static String key(int fromZone, int toZone, TimeBand timeBand) {
        return fromZone + "-" + toZone + "-" + timeBand.name();
    }

    private static BigDecimal money(String amount) {
        return new BigDecimal(amount).setScale(2, RoundingMode.HALF_UP);
    }

    private static Map<String, BigDecimal> buildBaseFare() {
        Map<String, BigDecimal> m = new HashMap<>();

        // Peak fares
        put(m,1,1,TimeBand.PEAK,"2.50"); put(m,1,2,TimeBand.PEAK,"3.20");
        put(m,1,3,TimeBand.PEAK,"3.80"); put(m,1,4,TimeBand.PEAK,"4.40");
        put(m,1,5,TimeBand.PEAK,"5.00");

        put(m,2,1,TimeBand.PEAK,"3.20"); put(m,2,2,TimeBand.PEAK,"2.30");
        put(m,2,3,TimeBand.PEAK,"3.10"); put(m,2,4,TimeBand.PEAK,"3.80");
        put(m,2,5,TimeBand.PEAK,"4.50");

        put(m,3,1,TimeBand.PEAK,"3.80"); put(m,3,2,TimeBand.PEAK,"3.10");
        put(m,3,3,TimeBand.PEAK,"2.10"); put(m,3,4,TimeBand.PEAK,"3.00");
        put(m,3,5,TimeBand.PEAK,"3.70");

        put(m,4,1,TimeBand.PEAK,"4.40"); put(m,4,2,TimeBand.PEAK,"3.80");
        put(m,4,3,TimeBand.PEAK,"3.00"); put(m,4,4,TimeBand.PEAK,"2.00");
```

```
        put(m,4,5,TimeBand.PEAK,"2.90");

        put(m,5,1,TimeBand.PEAK,"5.00"); put(m,5,2,TimeBand.PEAK,"4.50");
        put(m,5,3,TimeBand.PEAK,"3.70"); put(m,5,4,TimeBand.PEAK,"2.90");
        put(m,5,5,TimeBand.PEAK,"1.90");

        // Off-peak fares
        put(m,1,1,TimeBand.OFF_PEAK,"2.00");
put(m,1,2,TimeBand.OFF_PEAK,"2.70");
        put(m,1,3,TimeBand.OFF_PEAK,"3.20");
put(m,1,4,TimeBand.OFF_PEAK,"3.70");
        put(m,1,5,TimeBand.OFF_PEAK,"4.20");

        put(m,2,1,TimeBand.OFF_PEAK,"2.70");
put(m,2,2,TimeBand.OFF_PEAK,"1.90");
        put(m,2,3,TimeBand.OFF_PEAK,"2.60");
put(m,2,4,TimeBand.OFF_PEAK,"3.20");
        put(m,2,5,TimeBand.OFF_PEAK,"3.80");

        put(m,3,1,TimeBand.OFF_PEAK,"3.20");
put(m,3,2,TimeBand.OFF_PEAK,"2.60");
        put(m,3,3,TimeBand.OFF_PEAK,"1.70");
put(m,3,4,TimeBand.OFF_PEAK,"2.50");
        put(m,3,5,TimeBand.OFF_PEAK,"3.10");

        put(m,4,1,TimeBand.OFF_PEAK,"3.70");
put(m,4,2,TimeBand.OFF_PEAK,"3.20");
        put(m,4,3,TimeBand.OFF_PEAK,"2.50");
put(m,4,4,TimeBand.OFF_PEAK,"1.60");
        put(m,4,5,TimeBand.OFF_PEAK,"2.40");

        put(m,5,1,TimeBand.OFF_PEAK,"4.20");
put(m,5,2,TimeBand.OFF_PEAK,"3.80");
        put(m,5,3,TimeBand.OFF_PEAK,"3.10");
put(m,5,4,TimeBand.OFF_PEAK,"2.40");
        put(m,5,5,TimeBand.OFF_PEAK,"1.50");

        return Map.copyOf(m);
    }

    private static void put(Map<String, BigDecimal> m, int from, int to,
TimeBand band, String amount) {
        m.put(key(from, to, band), money(amount));
    }
}
```

References

Namma (n.d.). Namma-Metro-Trip-Planner. Retrieved from: https://github.com/mueezmamoon/Namma-Metro-Trip-Planner [Accessed 28 January 2026]

Oracle Corporation (n.d.). Class Scanner. Retrieved from: https://docs.oracle.com/javase/8/docs/api/java/util/Scanner.html [Accessed 5 February 2026].

GeeksforGeeks (2026, January). Classes and Objects in Java. Retrieved from: https://www.geeksforgeeks.org/java/classes-objects-java [Accessed 8 February 2026].

GeeksforGeeks (2026, February). ArrayList in Java. Retrieved from: https://www.geeksforgeeks.org/java/arraylist-in-java [Accessed 8 February 2026].

GeeksforGeeks (2025, July). BigDecimal divide() Method in Java with Examples. Retrieves from: https://www.geeksforgeeks.org/java/bigdecimal-divide-method-in-java-with-examples/ [Accessed 13 February 2026]

GeeksforGeeks (2025). Array of Objects in Java. Retrieved from: https://www.geeksforgeeks.org/array-of-objects-in-java/ [Accessed 13 February 2026].

GeeksforGeeks (2025). Encapsulation in Java. Retrieved from: https://www.geeksforgeeks.org/encapsulation-in-java/ [Accessed 13 February 2026].

Oracle (n.d.). Providing Constructors for Your Classes. Retrieved from: https://docs.oracle.com/javase/tutorial/java/javaOO/constructors.html [Accessed 13 February 2026].

Oracle (2014). Enum Types. Retrieved from: https://docs.oracle.com/javase/tutorial/java/javaOO/enum.html [Accessed 18 February 2026].

Appendices

IY4113 Milestone 1

Purpose of the Program

The program, which is a console-based Java application called CityRide Lite that allows the user to record and manage all of their public transport journeys for a single day. The user can enterjourney details such as starting zone, destination zone, time band (peak/off-peak), passenger type. With the use of the supplied City Ride dataset (fixed), the system calculates fares, discounts and daily caps.

Core program functionality

Display Menu: The program shows a main menu with different options such as adding journeys, viewing journeys, filtering, viewing summaries, and exiting. The menu keeps looping so the user can continue using the system until they choose to exit.

Add journeys: The user can add a new journey by entering the required details, including zones, time band, and passenger type. The system checks that the input is valid before saving the journey and confirming it has been added. Therefore;

- Assigns each journey a unique session IDs.
- Stores all journeys in memory for the current session.

Calculate Fares: The system Calculates how many zones were crossed using the formula abs (tozone – fromzone) + 1. From the provided CityRide dataset, the system will work out the base fare using: (from zone, to zone, time band) - > base fare.

Apply discounts:

- Adult: 0%
- Student: 25%
- Child: 50%
- Senior Citizen: 30%

Apply Daily Caps:

- Adult: £8.00
- Student: £6.00
- Child: £4.00
- Senior Citizen: £7.00

NOTE: Ensure fares never exceed the passenger's daily cap.

Store journeys: The program keeps a list of all journeys entered during the current session. Each journey stores information such as the date, starting and ending zones, passenger type, time band, and the final fare charged.

View journeys: The user can see all journeys that have been entered so far. Each journey is shown with its ID and key details so it is easy to understand and refer back to.

Filter journeys: The system allows the user to filter journeys by passenger type, time band, zone, or date. This makes it easier to look at specific trips instead of scrolling through everything.

Remove journeys: Similarly, If the user makes a mistake, they can remove a journey by ID, with confirmation. The program then updates the stored journeys and recalculates the totals.

Summary: The program produces summaries such as;

- Total number of journeys
- Total cost charged
- Average cost per journey
- Most expensive journey
- Totals per passenger type
- Peak vs Off-Peak counts
- Zone usage counts

System constraints (limitations / rules)

Single day session only: The system only stores journeys for one session and everything is

cleared when the program closes.

Console UI only: All interaction with the user happens through text menus and keyboard input.

Zones limited to 1 - 5: The user can only enter zones between 1 and 5, as these are the valid transport zones.

Fixed passenger types: Only Adult, Student, Child, and Senior Citizen are allowed so that discounts can be applied correctly.

Fixed time bands: Journeys must be either Peak or Off - Peak so the correct fare can be selected from the dataset.

Dataset rules cannot be changed: All fares, discounts, and caps must come from the provided CityRide dataset and cannot be edited or modified.

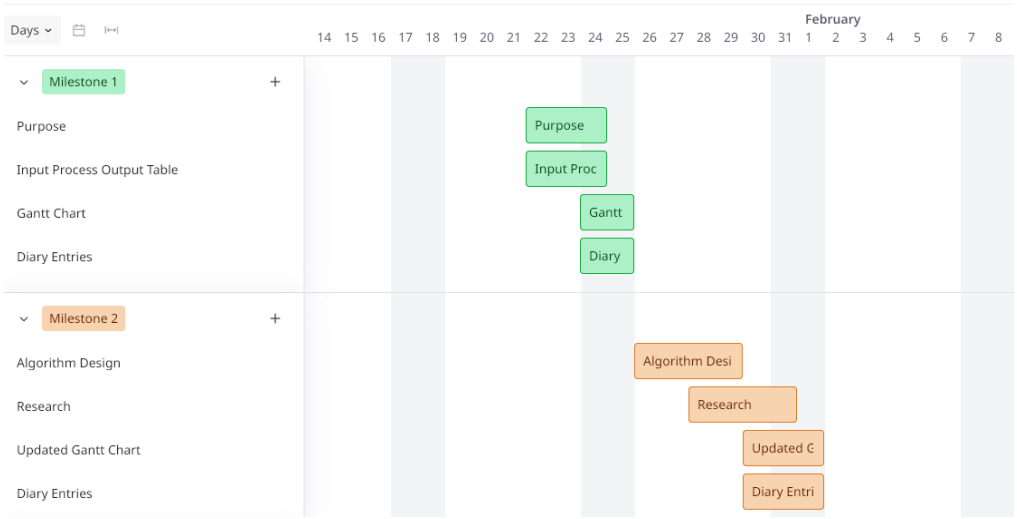
Daily caps apply per passenger type: Each passenger type has its own maximum amount that can be charged in one day.

No data is saved after exit: When the program is closed, all journeys and totals are lost and must be re-entered next time.

Input Process Output Table

Feature / Task	Inputs	Processing (what the system does)	Outputs
Start program	None (program launch)	Initialise data structures and running totals	Main menu shown
Display menu (FR1)	User menu choice (1 - 5) (int)	Validate choice and route to correct function	Selected screen displayed
Add journey (FR2)	Date(String), from zone(int), to zone(int), passenger type(String), time band(String) - peak/off-peak	Validate inputs, calculate zones crossed, lookup fare, apply discount and daily cap	Journey added confirmation
Calculate fare (FR3)	From zone (int), to zone(int), time band (String), passenger type (String)	Calculates zones crossed, get base fare, apply discount and daily cap	Final charged fare
Store journey (FR4)	Journey details (Data Structure)	Create Journey object and assign ID	Journey stored in memory
List journeys	View Request (Boolean: Y/N)	Loop through journey list	Display all journeys
Filter journeys (FR5)	Filter Criteria (Type: String, Value: String/Integer)	Search journey list	Filtered journey list
Remove journey (FR6)	Journey ID (Integer)	Validate ID, remove journey, recalculate totals	Confirmation or error
Daily summary (FR7)	Summary Request (Boolean: Y/N)	Calculate totals, averages, max fare	Summary report
Reset day	Confirmation (Boolean: Y/N)	Clear all lists and totals	System reset message
Exit	Menu choice – 5 : Exit Command (int)	Terminate program	Goodbye message

Gantt Chart



Diary Entries

23/01/2026 - Diary Entry 1 - Understanding the Problem and planning and Analysing of the System

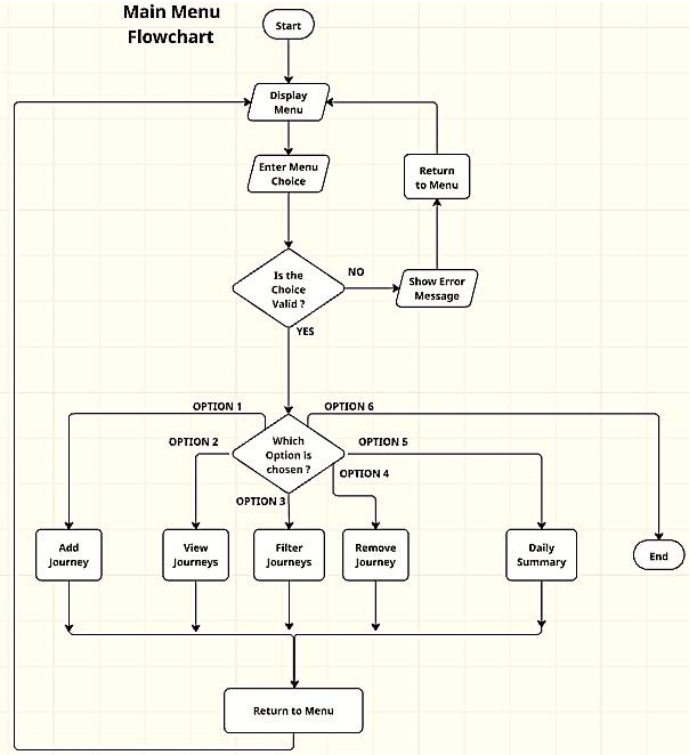
Today I focused on understanding the CityRide Lite specification and what the system was expected to do. I carefully read through the specification and highlighted the main features, such as adding journeys, calculating fares, applying discounts, and enforcing daily caps. After this, I wrote the purpose of the program in my own words so I had a clear idea of what I was trying to build. I created the Input Process Output Table to help map out how the user's input would be handled and turned into useful output by the system. This part of the work helped me break the problem into smaller pieces, which made it easier to understand. One challenge I had was making sure I included all the required inputs and outputs, especially for things like fare calculation and summaries. To deal with this, I went back to the brief and checked each requirement carefully against my table for thorough analysis.

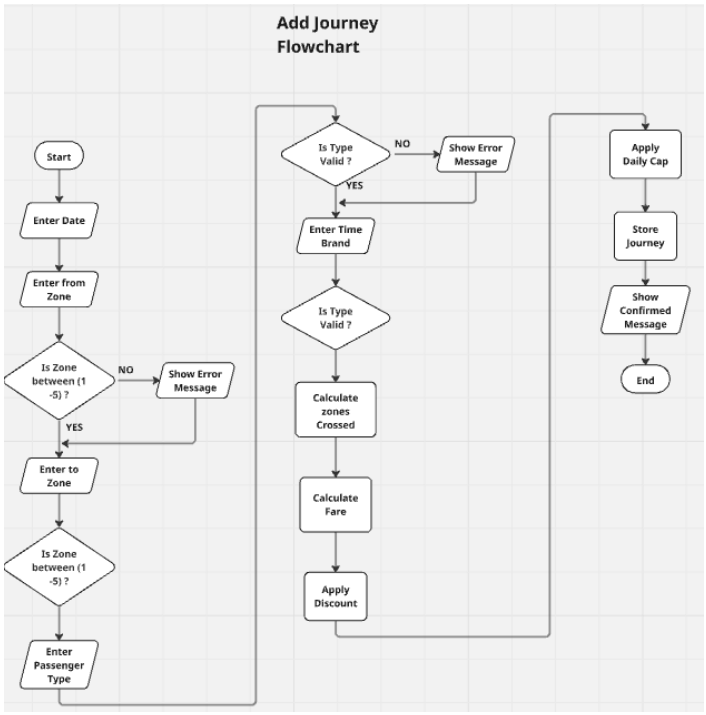
25/01/2026 - Diary Entry 2 - Planning and Creating the Gantt Chart

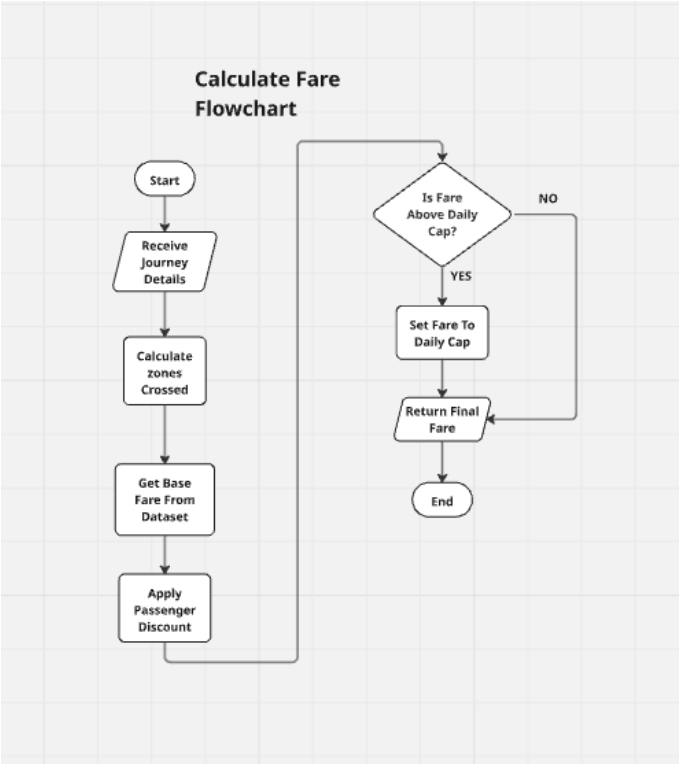
In this part of the milestone, I focused on organising the work for the whole CityRide Lite project. After completing the purpose of the program and the Input Process Output table, I used this information to create a Gantt chart. The chart shows all the numbered milestones and the final submission, so I could see how the work will be spread out over time. Even though only Milestone 1 has been completed so far, creating the Gantt chart still required me to think carefully about how the work would be spreaded over time and the later stages of the project. With this, I have a clear plan that I can follow as I move on to the next milestones.

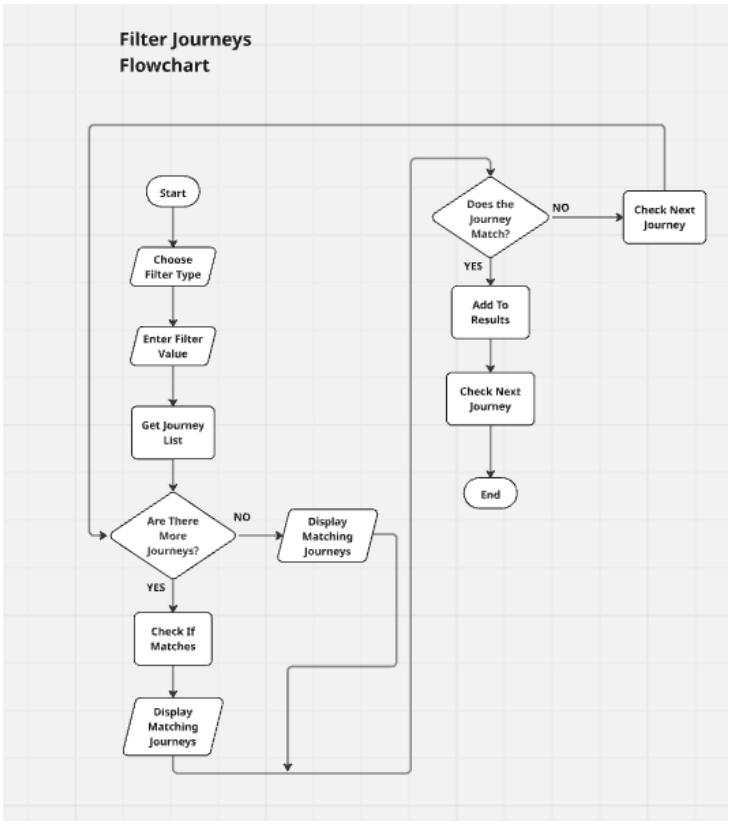
IY4113 Milestone 2

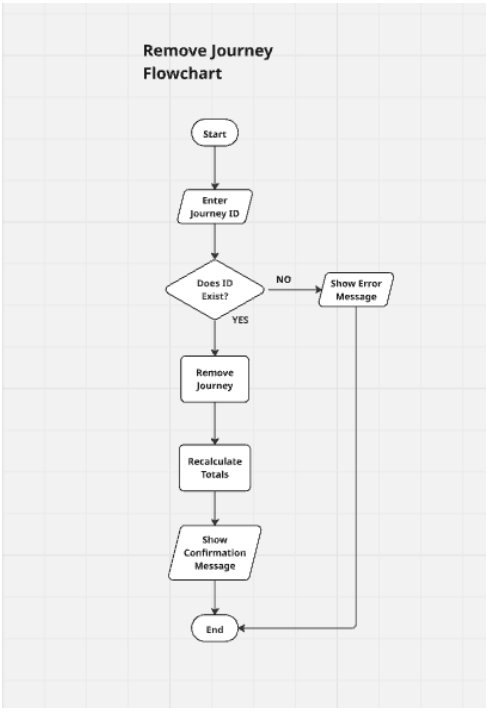
Algorithm Design



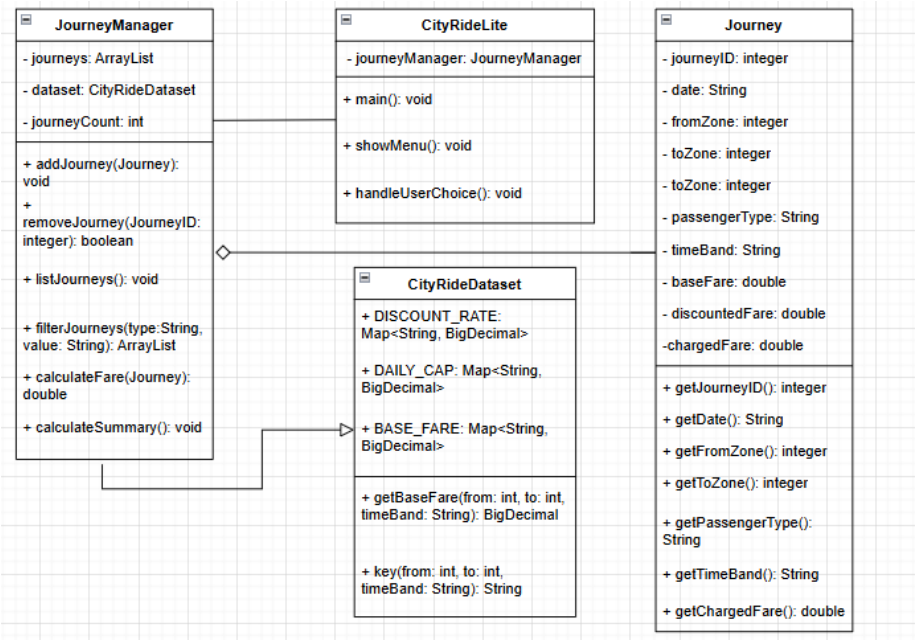

Add Journey
Flowchart
Start
Enter Date
Enter from Zone
Is Zone between (1 -5) ?
NO
Show Error Message
YES
Enter to Zone
Is Zone between (1 -5) ?
Enter Passenger Type
Is Type Valid ?
NO
Show Error Message
YES
Enter Time Brand
Is Type Valid ?
Calculate zones Crossed
Calculate Fare
Apply Discount
Apply Daily Cap
Store Journey
Show Confirmed Message
End


Calculate Fare
Flowchart
Start
Receive Journey Details
Calculate zones Crossed
Get Base Fare From Dataset
Apply Passenger Discount
Is Fare Above Daily Cap?
NO
YES
Set Fare To Daily Cap
Return Final Fare
End





Class Diagram



Research (minimum of 1 required, preferably 2)

The following research analyzes the Namma Metro Trip Planner, a Java based console application. This program serves as a great technical reference for transport systems, demonstrating how to effectively manages station data through arrays and figures out complicated routes within a command line environment.

Name of program: Namma Metro Trip Planner

Reference (link): Namma (n.d.). Namma-Metro-Trip-Planner. Retrieved from: https://github.com/mueezmamoon/Namma-Metro-Trip-Planner [Accessed 28 January 2026]

What it does well (2-3 features that work effectively):

The program effectively uses String arrays (purp[] and green[]) to store station names. This allows for easy lookup and print of the journey path, similar to how the CityRide Lite dataset stores base fares. It doesn't just calculate a fare; it determines the direction of travel (e.g., "Board trains going towards Nagasandra") and the platform number. This feature adds a high level of utility for the user. It handles "Line Selection" and "Interchanges" (Majestic Station) effectively using boolean flags like changeLines = true, showing how to manage state across different parts of a journey.

What it does poorly (at least 1 feature):

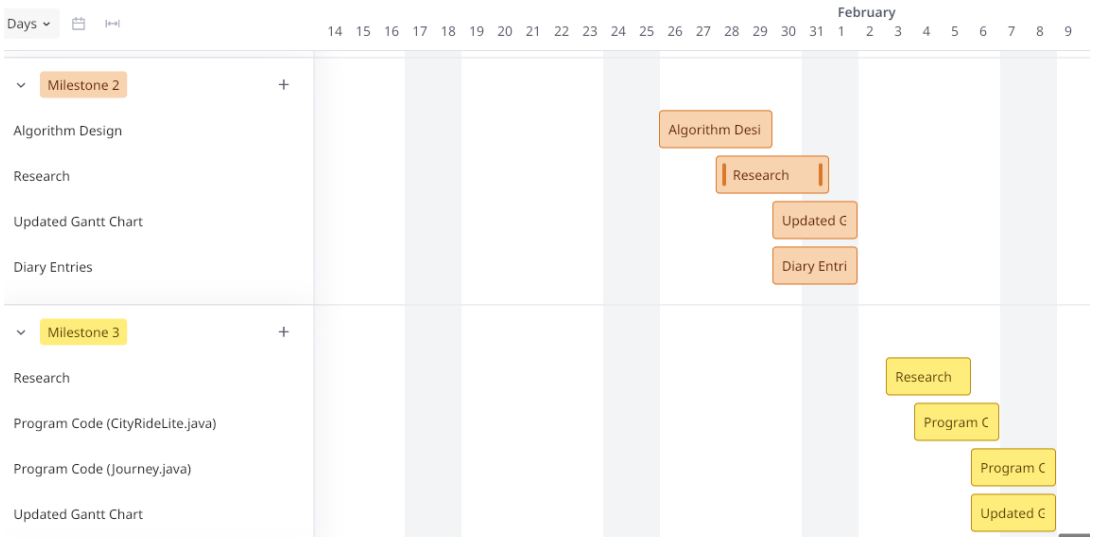
The logic is heavily nested with if/else statements within a single main class. This makes the code difficult to maintain and therefore, lacks the Functional Decomposition.

Key design ideas you could reuse (e.g., layout, navigation, input/output, program structure):

The use of a numbered menu for line selection (e.g., "1. Purple Line", "2. Green Line") is a clean way to handle user input. The program uses text-based lines (e.g., ----------) to separate different parts of the console output, which makes the interface much easier to read. The while loops used to print the sequence of stations passed (System.out.print("

"+green[transitDepart]+" -->")) could be adapted to my View Journeys feature to show the zones crossed.

Updated Gantt Chart



Diary Entries

27/01/2026 - Diary Entry 1 – Algorithm Design

In this stage of the project, I focused on designing the algorithms for the CityRide Lite system using flowcharts. I started by breaking the program down into smaller parts, such as the main menu, adding a journey, calculating fares, filtering journeys, and removing journeys. This helped me to clearly understand how the system should flow and how the user will interact with it at each step. Using flowcharts also made it easier to think about validation, such as checking if zones are within the correct range and ensuring passenger types and time bands are valid before continuing. I also further created Class diagram for the project.

01/02/2026 - Diary Entry 2 - Research and Gantt Chart Update

Today I carried out research on an existing console-based transport system called the Namma Metro Trip Planner to understand how similar problems have been approached in practice. This helped me see how station data can be stored using arrays and how user input can be handled through menus and simple logic. I found some useful design ideas, such as using numbered menu options and clear text separators to improve readability in the console. I also noticed that the program relied heavily on nested if/else statements, which made the structure harder to follow, and this reinforced the importance of using better functional decomposition in my own design. After completing the research, I updated the Gantt chart to reflect the planned design and implementation tasks for the remaining milestones.

IY4113 Milestone 3

Research (minimum of 2, at least 3)

The following researches focus on Java programming concepts and resources that support the development of the CityRide Lite application. It explores official documentation and tutorials related to user input handling, object - oriented programming, and data structures. These resources helped guide coding decisions, improve program structure, and ensure that the implementation follows good Java programming practices within a console - based environment.

These include the following;

Research 1

Title of research: Java Scanner Class Tutorial (Oracle Documentation)

Reference (link): Oracle Corporation (n.d.). Class Scanner. Retrieved from: https://docs.oracle.com/javase/8/docs/api/java/util/Scanner.html [Accessed 8 February 2026].

How does the research help with coding practise?

This research helped me understand how to correctly take user input in a Java console application using the Scanner class. Since my program is menu based and requires the user to enter values such as menu choices, zones, passenger type and time band, this documentation was important for learning how to use methods like nextInt() and next() properly. It also helped me understand common issues such as input mismatch errors and how to avoid problems when switching between reading numbers and strings. Overall, it made my input handling more reliable and reduced bugs in the program.

Key coding ideas you could reuse in your program:

- Using Scanner to read user input from the keyboard.
- Using nextInt() for numeric input such as menu options and zones.
- Using next() or nextLine() for text input such as passenger type.
- Reusing a single Scanner object instead of creating many.

Screenshot of research:

Class Scanner

java.lang.Object
    java.util.Scanner

All Implemented Interfaces:
`Closeable, AutoCloseable, Iterator<String>`

```
public final class Scanner
extends Object
implements Iterator<String>, Closeable
```

A simple text scanner which can parse primitive types and strings using regular expressions.

A `Scanner` breaks its input into tokens using a delimiter pattern, which by default matches whitespace. The resulting tokens may then be
methods.

For example, this code allows a user to read a number from `System.in`:

```
Scanner sc = new Scanner(System.in);
int i = sc.nextInt();
```

As another example, this code allows `long` types to be assigned from entries in a file `myNumbers`:

```
Scanner sc = new Scanner(new File("myNumbers"));
while (sc.hasNextLong()) {
    long aLong = sc.nextLong();
}
```

Research 2

Title of research: Classes and Objects in Java (GeeksforGeeks)

Reference (link): GeeksforGeeks (2026, January). Classes and Objects in Java. Retrieved from: https://www.geeksforgeeks.org/java/classes-objects-java [Accessed 8 February 2026].

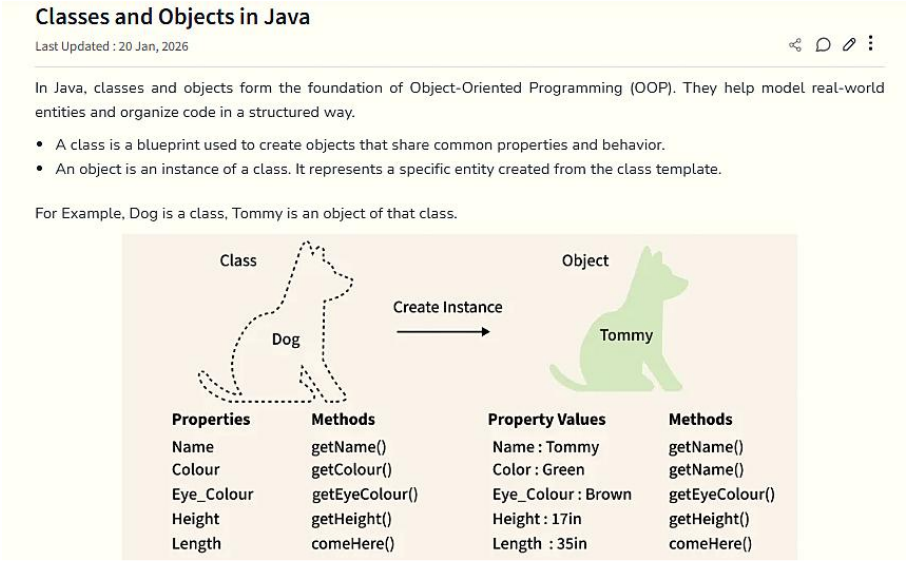
How does the research help with coding practise ?

In this research, I understood the basic principles of object - oriented programming in Java, especially creating classes, defining attributes, and creating objects. It supported the design of my program by helping me separate my code into different classes such as Journey, JourneyManager, and CityRideLite instead of placing everything inside one main class. This made the program easier to understand, easier to maintain, and easier to extend in later milestones when more features are added.

Key coding ideas you could reuse in your program:

- Creating classes to represent real world objects (e.g. a Journey).
- Using constructors to initialise object data.
- Using getters and setters to access private fields.
- Keeping each class responsible for one main task.
- Following the idea of encapsulation by hiding internal data.

Screenshot of research:



Classes and Objects in Java

Last Updated : 20 Jan, 2026

In Java, classes and objects form the foundation of Object-Oriented Programming (OOP). They help model real-world entities and organize code in a structured way.

- A class is a blueprint used to create objects that share common properties and behavior.
- An object is an instance of a class. It represents a specific entity created from the class template.

For Example, Dog is a class, Tommy is an object of that class.

Properties	Methods	Property Values	Methods
Name	getName()	Name : Tommy	getName()
Colour	getColour()	Color : Green	getName()
Eye_Colour	getEyeColour()	Eye_Colour : Brown	getEyeColour()
Height	getHeight()	Height : 17in	getHeight()
Length	comeHere()	Length : 35in	comeHere()

Research 3

Title of research: Java ArrayList (GeeksforGeeks)

Reference (link): GeeksforGeeks (2026, February). ArrayList in Java. Retrieved from: https://www.geeksforgeeks.org/java/arraylist-in-java [Accessed 8 February 2026].

How does the research help with coding practise ?

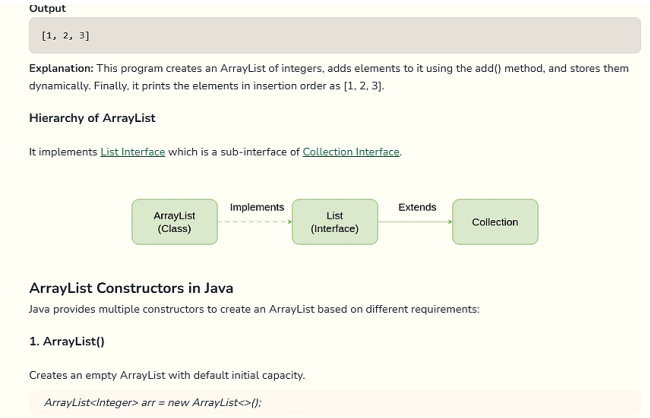
This research explains how the ArrayList data structure works in Java and how it can be used to store and manage collections of objects. Although my current version of the program uses simple variables, this research showed me how I can later store multiple Journey objects inside an ArrayList. This will be useful for features such as listing journeys, filtering journeys and generating summaries, because ArrayLists allow the program to grow dynamically as more journeys are added.

Key coding ideas you could reuse in your program:

- Using an ArrayList to store Journey objects.
- Adding new journeys using the .add() method.
- Looping through the list using a for loop or enhanced for loop.
- Using .size() to calculate totals and summaries.
- Removing items from the list when a journey is deleted.

Screenshot of research:

```
#geeksforgeeks
import ArrayList;
class Main {
public static void main (String[]
args) {
// Creating an ArrayList
ArrayList<Integer> a = new
ArrayList<Integer>();
// Adding Element in ArrayList
a.add(1);
a.add(2);
a.add(3);
// Printing ArrayList
System.out.println(a);
}
}
java.util
```



Program Code

CityRideLite.java (Main Class)

This is the main entry point of the program and is responsible for controlling the overall flow of the application. It displays the main menu, handles user input, and calls the appropriate methods based on the user's selection. The class uses a while loop to keep the program running until the user chooses to exit. This class acts as the user interface controller, managing how the user interacts with the system through the console.

```
import Scanner;
public class CityRideLite {
public static void main(String[] args) {
Scanner scanner = new Scanner(System.in);
JourneyManager journeyManager = new JourneyManager();
int choice;
do {
System.out.println("\n--- CityRide Lite ---");
System.out.println("1. Add Journey");
System.out.println("2. View Journeys");
System.out.println("3. Exit");
System.out.print("Enter choice: ");
choice = scanner.nextInt();
if (choice == 1) {
System.out.print("Enter from zone (1-5): ");
int fromZone = scanner.nextInt();
System.out.print("Enter to zone (1-5): ");
int toZone = scanner.nextInt();
System.out.print("Enter passenger type: ");
String passengerType = scanner.next();
System.out.print("Enter time band: ");
String timeBand = scanner.next();
Journey journey = new Journey(fromZone, toZone, passengerType,
timeBand);
journeyManager.addJourney(journey);
} else if (choice == 2) {
```

```
journeyManager.listJourneys();
} else if (choice == 3) {
System.out.println("Exiting program...");
} else {
System.out.println("Invalid option.");
}
} while (choice != 3);
}
}
```

java.util

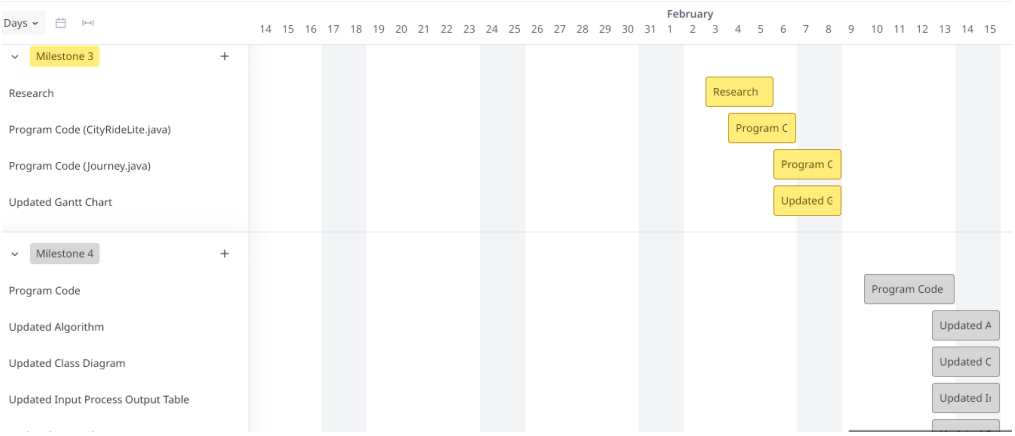
Journey.java (Data Class)

The Journey class represents a single public transport journey. It stores all the relevant information about a trip, such as the journey ID, starting zone, destination zone, passenger type, and time band. Each time a new journey is created.

```
public class Journey {
private int fromZone;
private int toZone;
private String passengerType;
private String timeBand;
public Journey(int fromZone, int toZone, String passengerType, String timeBand) {
this.fromZone = fromZone;
this.toZone = toZone;
this.passengerType = passengerType;
this.timeBand = timeBand;
}
public int getFromZone() {
return fromZone;
}
public int getToZone() {
return toZone;
}
public String getPassengerType() {
return passengerType;
}
public String getTimeBand() {
return timeBand;
}
public void displayJourney() {
System.out.println("From Zone: " + fromZone);
System.out.println("To Zone: " + toZone);
System.out.println("Passenger Type: " + passengerType);
System.out.println("Time Band: " + timeBand);
}

}
```

Updated Gantt Chart



Diary Entries

05/02/2026 - Diary Entry 1 – Research and Initial Program Structure

In this milestone, I focused on conducting research to support my coding process and understanding how to properly structure a Java console application. I used resources such as the Oracle documentation for the Scanner class and GeeksforGeeks tutorials on object – oriented programming and ArrayLists. This research helped me understand how to handle user input, create classes, and organise data in a more structured way, which influenced how I designed the main menu and input logic for the CityRide Lite program. In addition, I started with implementing the main program file (CityRideLite.java). This class currently handles displaying the menu, taking basic user input, and controlling the overall flow of the program. Although the functionality is still limited, this milestone helped me establish the core structure of the system and provided a foundation that I can build on in later milestones.

08/02/2026 - Diary Entry 2 – Additional Program Class and Gantt Chart Update

Today, I created the core data class (Journey.java) to represent a single transport journey. This class stores information such as the starting zone, destination zone, passenger type, and time band. Creating this class helped me apply object - oriented principles by separating data representation from program logic, which makes the system easier to manage and extend. I also updated the Gantt chart to reflect the planned work for Milestones 3. The JourneyManager class and the provided dataset (CityRideDataset.java) will be implemented in the next milestone as the program functionality is which includes implementing journey management, fare calculations, and summary features.

IY4113 Milestone 4

Program Code

CityRideLite.java (Main Class)

This is the main entry point of the program and is responsible for controlling the overall flow of the application. It displays the main menu, handles user input, and calls the appropriate methods based on the user's selection. The class uses a while loop to keep the program running until the user chooses to exit. This class acts as the user interface controller, managing how the user interacts with the system through the console.

```
import Scanner;
```

```
public class CityRideLite {
public static void main(String[] args) {
Scanner scanner = new Scanner(System.in);
JourneyManager journeyManager = new JourneyManager();
int choice;
do {
System.out.println("\n--- CityRide Lite ---");
System.out.println("1. Add Journey");
System.out.println("2. View Journeys");
System.out.println("3. Filter Journeys");
System.out.println("4. Remove Journey");
System.out.println("5. Daily Summary");
System.out.println("6. Exit");
System.out.print("Enter choice: ");
choice = scanner.nextInt();
if (choice == 1) {
int fromZone;
int toZone;
do {
System.out.print("Enter from zone (1-5): ");
fromZone = scanner.nextInt();
} while (fromZone < 1 || fromZone > 5);
do {
System.out.print("Enter to zone (1-5): ");
toZone = scanner.nextInt();
} while (toZone < 1 || toZone > 5);
System.out.print("Passenger type (ADULT/STUDENT/CHILD/SENIOR_CITIZEN):
");
CityRideDataset.PassengerType passengerType =
CityRideDataset.PassengerType.valueOf(
scanner.next().toUpperCase()
);
System.out.print("Time band (PEAK/OFF_PEAK): ");
CityRideDataset.TimeBand timeBand =
CityRideDataset.TimeBand.valueOf(
scanner.next().toUpperCase()
);
journeyManager.addJourney(fromZone, toZone, passengerType, timeBand);
else if (choice == 2) {
journeyManager.listJourneys();
} else if (choice == 3) {
System.out.print("Filter by passenger type: ");
CityRideDataset.PassengerType type =
CityRideDataset.PassengerType.valueOf(
scanner.next().toUpperCase()
);
journeyManager.filterByPassengerType(type);
} else if (choice == 4) {
System.out.print("Enter Journey ID to remove: ");
int id = scanner.nextInt();
journeyManager.removeJourney(id);
} else if (choice == 5) {
journeyManager.showSummary();
} else if (choice == 6) {
System.out.println("Exiting program...");
} else {
System.out.println("Invalid option.");
}
```

```
} while (choice != 6);
scanner.close();
}

}
```

Journey.java (Data Class)

The Journey class represents a single public transport journey. It stores all the relevant information about a trip, such as the journey ID, starting zone, destination zone, passenger type, and time band. Each time a new journey is created.

```
import BigDecimal;
public class Journey {
private int id;
private int fromZone;
private int toZone;
private CityRideDataset.PassengerType passengerType;
private CityRideDataset.TimeBand timeBand;
private BigDecimal fare;
public Journey(int id, int fromZone, int toZone,
CityRideDataset.PassengerType passengerType,
CityRideDataset.TimeBand timeBand,
BigDecimal fare) {
this.id = id;
this.fromZone = fromZone;
this.toZone = toZone;
this.passengerType = passengerType;
this.timeBand = timeBand;
this.fare = fare;
}
public int getId() {
return id;
}
public int getFromZone() {
return fromZone;
}
public int getToZone() {
return toZone;
}
public CityRideDataset.PassengerType getPassengerType() {
return passengerType;
}
public CityRideDataset.TimeBand getTimeBand() {
return timeBand;
}
public BigDecimal getFare() {
return fare;
}
public void setFare(BigDecimal fare) {
this.fare = fare;
}
public void displayJourney() {
System.out.println("ID: " + id);
System.out.println("From Zone: " + fromZone);
System.out.println("To Zone: " + toZone);
System.out.println("Passenger Type: " + passengerType);
```

```
System.out.println("Time Band: " + timeBand);
System.out.println("Fare: £" + fare);
}

}
```

JourneyManager.java (Data Class)

The JourneyManager class manages all journey related operations, including storing journeys, removing journeys, filtering journeys, and generating daily summaries. It also handles fare calculations by using the rules provided in the CityRideDataset class.

GeeksforGeeks (2025, July). BigDecimal divide() Method in Java with Examples. Retrieves from: https://www.geeksforgeeks.org/java/bigdecimal-divide-method-in-java-with-examples/ [Accessed 13 February 2026]

```
import BigDecimal;
import RoundingMode;
public class JourneyManager {
private Journey[] journeys;
private int journeyCount;
private int nextId;
public JourneyManager() {
journeys = new Journey[5];
journeyCount = 0;
nextId = 1;
}
public void addJourney(int fromZone, int toZone,
CityRideDataset.PassengerType passengerType,
CityRideDataset.TimeBand timeBand) {
BigDecimal baseFare =
CityRideDataset.getBaseFare(fromZone, toZone, timeBand);
BigDecimal discountRate =
CityRideDataset.DISCOUNT_RATE.get(passengerType);
BigDecimal discount =
baseFare.multiply(discountRate);
BigDecimal finalFare =
baseFare.subtract(discount);
BigDecimal totalSoFar = getTotalForPassenger(passengerType);
BigDecimal cap = CityRideDataset.DAILY_CAP.get(passengerType);
if (totalSoFar.add(finalFare).compareTo(cap) > 0) {
finalFare = cap.subtract(totalSoFar);
if (finalFare.compareTo(BigDecimal.ZERO) < 0) {
finalFare = BigDecimal.ZERO;
}
}
Journey journey = new Journey(
nextId,
fromZone,
toZone,
passengerType,
timeBand,
finalFare
);
java.math.
java.math.
journeys[journeyCount] = journey;
```

```
journeyCount++;
nextId++;
System.out.println("Journey added successfully.");
System.out.println("Fare charged: £" + finalFare);
}
private BigDecimal getTotalForPassenger(CityRideDataset.PassengerType type) {
BigDecimal total = BigDecimal.ZERO;
for (int i = 0; i < journeyCount; i++) {
if (journeys[i].getPassengerType() == type) {
total = total.add(journeys[i].getFare());
}
}
return total;
}
public void listJourneys() {
if (journeyCount == 0) {
System.out.println("No journeys recorded.");
} else {
for (int i = 0; i < journeyCount; i++) {
System.out.println();
journeys[i].displayJourney();
}
}
}
public void removeJourney(int id) {
boolean found = false;
for (int i = 0; i < journeyCount; i++) {
if (journeys[i].getId() == id) {
found = true;
for (int j = i; j < journeyCount - 1; j++) {
journeys[j] = journeys[j + 1];
}
journeyCount--;
System.out.println("Journey removed.");
break;
}
}
if (!found) {
System.out.println("Journey ID not found.");
}
}
public void filterByPassengerType(CityRideDataset.PassengerType type) {
boolean found = false;
for (int i = 0; i < journeyCount; i++) {
if (journeys[i].getPassengerType() == type) {
journeys[i].displayJourney();
found = true;
System.out.println();
}
}
if (!found) {
System.out.println("No journeys found for that passenger type.");
}
}
public void showSummary() {
if (journeyCount == 0) {
System.out.println("No journeys to summarise.");
return;
```

```
}
BigDecimal total = BigDecimal.ZERO;
BigDecimal max = BigDecimal.ZERO;
int peakCount = 0;
int offPeakCount = 0;
for (int i = 0; i < journeyCount; i++) {
total = total.add(journeys[i].getFare());
if (journeys[i].getFare().compareTo(max) > 0) {
max = journeys[i].getFare();
}
if (journeys[i].getTimeBand() == CityRideDataset.TimeBand.PEAK) {
peakCount++;
} else {
offPeakCount++;
}
}
BigDecimal average =
total.divide(BigDecimal.valueOf(journeyCount), 2,
RoundingMode.HALF_UP);
System.out.println("\n--- Daily Summary ---");
System.out.println("Total journeys: " + journeyCount);
System.out.println("Total cost: £" + total);
System.out.println("Average cost: £" + average);
System.out.println("Most expensive journey: £" + max);
System.out.println("Peak journeys: " + peakCount);
System.out.println("Off - peak journeys: " + offPeakCount);
}

}
```

CityRideDataset.java

The CityRideDataset class is a utility class that contains all fixed fare data, discounts, and daily caps. It provides static methods that allow the system to retrieve base fares and apply rules consistently.

```
import BigDecimal;
import RoundingMode;
import HashMap;
import Map;
final class CityRideDataset {
private CityRideDataset() {}
public static final int MIN_ZONE = 1;
public static final int MAX_ZONE = 5;
public enum TimeBand {
PEAK,
OFF_PEAK
}
public enum PassengerType {
ADULT,
STUDENT,
CHILD,
SENIOR_CITIZEN
}
public static final Map<PassengerType, BigDecimal> DISCOUNT_RATE = Map.of(
PassengerType.ADULT, new BigDecimal("0.00"),
PassengerType.STUDENT, new BigDecimal("0.25"),
```

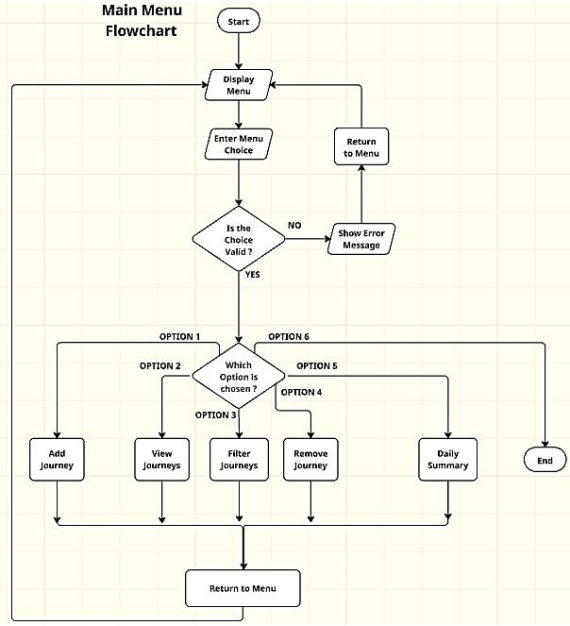
```
PassengerType.CHILD, new BigDecimal("0.50"),
PassengerType.SENIOR_CITIZEN, new BigDecimal("0.30")
);
public static final Map<PassengerType, BigDecimal> DAILY_CAP = Map.of(
PassengerType.ADULT, new BigDecimal("8.00"),
PassengerType.STUDENT, new BigDecimal("6.00"),
PassengerType.CHILD, new BigDecimal("4.00"),
PassengerType.SENIOR_CITIZEN, new BigDecimal("7.00")
);
public static final Map<String, BigDecimal> BASE_FARE = buildBaseFare();
public static BigDecimal getBaseFare(int fromZone, int toZone, TimeBand timeBand)
{
return BASE_FARE.get(key(fromZone, toZone, timeBand));
}
public static String key(int fromZone, int toZone, TimeBand timeBand) {
return fromZone + "-" + toZone + "-" + timeBand.name();
}
private static BigDecimal money(String amount) {
return new BigDecimal(amount).setScale(2, RoundingMode.HALF_UP);
}
private static Map<String, BigDecimal> buildBaseFare() {
Map<String, BigDecimal> m = new HashMap<>();
// Peak fares
put(m,1,1,TimeBand.PEAK,"2.50"); put(m,1,2,TimeBand.PEAK,"3.20");
put(m,1,3,TimeBand.PEAK,"3.80"); put(m,1,4,TimeBand.PEAK,"4.40");
put(m,1,5,TimeBand.PEAK,"5.00");
put(m,2,1,TimeBand.PEAK,"3.20"); put(m,2,2,TimeBand.PEAK,"2.30");
put(m,2,3,TimeBand.PEAK,"3.10"); put(m,2,4,TimeBand.PEAK,"3.80");
put(m,2,5,TimeBand.PEAK,"4.50");
put(m,3,1,TimeBand.PEAK,"3.80"); put(m,3,2,TimeBand.PEAK,"3.10");
put(m,3,3,TimeBand.PEAK,"2.10"); put(m,3,4,TimeBand.PEAK,"3.00");
put(m,3,5,TimeBand.PEAK,"3.70");
put(m,4,1,TimeBand.PEAK,"4.40"); put(m,4,2,TimeBand.PEAK,"3.80");
put(m,4,3,TimeBand.PEAK,"3.00"); put(m,4,4,TimeBand.PEAK,"2.00");
put(m,4,5,TimeBand.PEAK,"2.90");
put(m,5,1,TimeBand.PEAK,"5.00"); put(m,5,2,TimeBand.PEAK,"4.50");
put(m,5,3,TimeBand.PEAK,"3.70"); put(m,5,4,TimeBand.PEAK,"2.90");
put(m,5,5,TimeBand.PEAK,"1.90");
// Off-peak fares
put(m,1,1,TimeBand.OFF_PEAK,"2.00"); put(m,1,2,TimeBand.OFF_PEAK,"2.70");
put(m,1,3,TimeBand.OFF_PEAK,"3.20"); put(m,1,4,TimeBand.OFF_PEAK,"3.70");
put(m,1,5,TimeBand.OFF_PEAK,"4.20");
put(m,2,1,TimeBand.OFF_PEAK,"2.70"); put(m,2,2,TimeBand.OFF_PEAK,"1.90");
put(m,2,3,TimeBand.OFF_PEAK,"2.60"); put(m,2,4,TimeBand.OFF_PEAK,"3.20");
put(m,2,5,TimeBand.OFF_PEAK,"3.80");
put(m,3,1,TimeBand.OFF_PEAK,"3.20"); put(m,3,2,TimeBand.OFF_PEAK,"2.60");
put(m,3,3,TimeBand.OFF_PEAK,"1.70"); put(m,3,4,TimeBand.OFF_PEAK,"2.50");
put(m,3,5,TimeBand.OFF_PEAK,"3.10");
put(m,4,1,TimeBand.OFF_PEAK,"3.70"); put(m,4,2,TimeBand.OFF_PEAK,"3.20");
put(m,4,3,TimeBand.OFF_PEAK,"2.50"); put(m,4,4,TimeBand.OFF_PEAK,"1.60");
put(m,4,5,TimeBand.OFF_PEAK,"2.40");
put(m,5,1,TimeBand.OFF_PEAK,"4.20"); put(m,5,2,TimeBand.OFF_PEAK,"3.80");
put(m,5,3,TimeBand.OFF_PEAK,"3.10"); put(m,5,4,TimeBand.OFF_PEAK,"2.40");
put(m,5,5,TimeBand.OFF_PEAK,"1.50");
return Map.copyOf(m);
}
private static void put(Map<String, BigDecimal> m, int from, int to, TimeBand
band, String amount) {
```

```
m.put(key(from, to, band), money(amount));
}

}
```

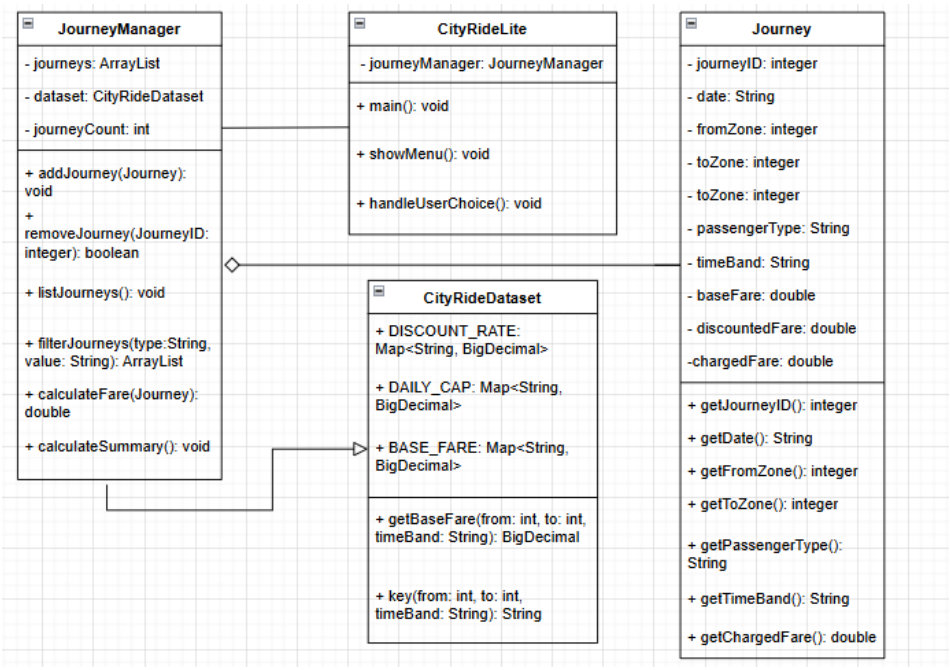
Updated Algorithm

This is the updated Algorithm flowchart for MainMenu.



Updated Class Diagram

This is the updated Class Diagram

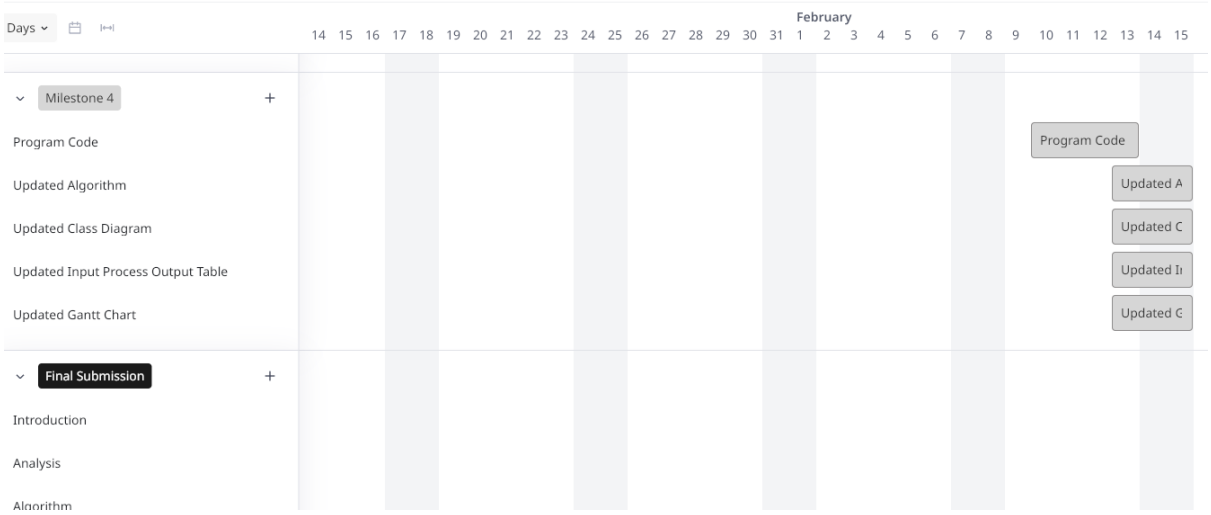


Input Process Output Table

This is the update for the input Process Output Table with the appropriate data types of the inputs.

Feature / Task	Inputs	Processing (what the system does)	Outputs
Start program	None (program launch)	Initialise data structures and running totals	Main menu shown
Display menu (FR1)	User menu choice (1 - 5) (int)	Validate choice and route to correct function	Selected screen displayed
Add journey (FR2)	Date(String), from zone(int), to zone(int), passenger type(String), time band(String) - peak/off-peak	Validate inputs, calculate zones crossed, lookup fare, apply discount and daily cap	Journey added confirmation
Calculate fare (FR3)	From zone (int), to zone(int), time band (String), passenger type (String)	Calculates zones crossed, get base fare, apply discount and daily cap	Final charged fare
Store journey (FR4)	Journey details (Data Structure)	Create Journey object and assign ID	Journey stored in memory
List journeys	View Request (Boolean: Y/N)	Loop through journey list	Display all journeys
Filter journeys (FR5)	Filter Criteria (Type: String, Value: String/Integer)	Search journey list	Filtered journey list
Remove journey (FR6)	Journey ID (Integer)	Validate ID, remove journey, recalculate totals	Confirmation or error
Daily summary (FR7)	Summary Request (Boolean: Y/N)	Calculate totals, averages, max fare	Summary report
Reset day	Confirmation (Boolean: Y/N)	Clear all lists and totals	System reset message
Exit	Menu choice – 5 : Exit Command (int)	Terminate program	Goodbye message

Updated Gantt Chart



Diary Entries

13/02/2026 Diary Entry 1 – Program Implementation

In this part of the milestone, I focused mainly on completing the core program code for the CityRide Lite system. I implemented the JourneyManager class to handle the storage and management of journeys, including adding, listing, filtering, and removing journeys. I also integrated the provided CityRideDataset class into the project as a separate utility class to ensure that all fares, discounts, and daily caps are calculated using the fixed dataset rules.

15/02/2026 Diary Entry 2 – Algorithm Design Updates and Gantt Chart

After completing most of the coding, I reviewed and updated the design documents to match the final implementation. This included improving the main menu flowchart, refining the class diagram to show clearer relationships between classes, correcting the Input Process Output table based on lecturer feedback, and updating the Gantt chart to reflect the progress made in Milestone 4.